

Arquitetando para a nuvem

Melhores práticas da AWS

Fevereiro de 2016



© 2016, Amazon Web Services, Inc. ou suas afiliadas. Todos os direitos reservados.

Avisos

Este documento é fornecido apenas para fins informativos. Ele relaciona as atuais ofertas de produtos e práticas da AWS na data de emissão deste documento, que estão sujeitas a alterações sem aviso prévio. Os clientes são responsáveis por fazer sua própria avaliação independente das informações neste documento e de qualquer uso dos produtos ou serviços da AWS, cada um dos quais é fornecido “como está”, sem garantia de qualquer tipo, expressa ou implícita. Este documento não cria quaisquer garantias, representações, compromissos contratuais, condições ou promessas da AWS, suas afiliadas, fornecedores ou licenciadores. As responsabilidades e obrigações da AWS para com seus clientes são controladas por contratos da AWS, e este documento não modifica nem faz parte de qualquer contrato entre a AWS e seus clientes.

Sumário

Resumo	4
Introdução	4
A diferença da computação em nuvem	5
Ativos de TI se tornam recursos programáveis	5
Global, disponível e capacidade ilimitada	5
Serviços gerenciados de nível superior	5
Segurança integrada	6
Princípios do design	6
Escalabilidade	6
Recursos descartáveis em vez de servidores fixos	11
Automation	15
Baixo acoplamento	16
Serviços, não servidores	21
Bancos de dados	23
Removendo pontos únicos de falha	29
Otimizar o custo	34
Armazenamento em cache	38
Segurança	39
Conclusão	42
Contribuidores	43
Outras leituras	43
Observações	44

Resumo

Este whitepaper destina-se a arquitetos e desenvolvedores de soluções que estejam criando soluções que serão implantadas na Amazon Web Services (AWS). Ele fornece padrões de arquitetura e conselhos sobre como projetar sistemas que sejam seguros, confiáveis, de alto desempenho e econômicos. Ele inclui uma discussão sobre como aproveitar os atributos específicos da natureza dinâmica da computação em nuvem (elasticidade, automação de infraestrutura, etc.). Além disso, este whitepaper também aborda padrões gerais, explicando como eles estão evoluindo e como são aplicados no contexto da computação em nuvem.

Introdução

A migração de aplicativos para a AWS, mesmo sem alterações significativas (uma abordagem conhecida como “lift and shift”), oferece às organizações os benefícios de uma infraestrutura segura e econômica. No entanto, para aproveitar ao máximo a elasticidade e a agilidade possíveis com a computação em nuvem, os engenheiros terão que desenvolver suas arquiteturas para aproveitar os recursos da AWS.

Para novos aplicativos, os clientes da AWS têm descoberto padrões de arquitetura de TI específicos para a nuvem, gerando ainda mais eficiência e escalabilidade. Essas novas arquiteturas podem suportar desde analítica em tempo real de dados em escala da Internet até aplicativos com tráfego imprevisível de milhares de Internet das Coisas (IoT) conectadas ou dispositivos móveis.

Este documento destacará os princípios a serem considerados se você estiver migrando aplicativos existentes para a AWS ou projetando novos aplicativos para a nuvem.

Este whitepaper pressupõe um conhecimento básico dos serviços e das soluções da AWS. Se você é novo na AWS, primeiro consulte a página [Sobre a AWS](#)¹.

A diferença da computação em nuvem

Esta seção analisa como a computação em nuvem difere de um ambiente tradicional e por que essas novas melhores práticas surgiram.

Ativos de TI se tornam recursos programáveis

Em um ambiente sem nuvem, você teria que provisionar a capacidade com base em uma suposição de um pico máximo teórico. Isso pode resultar em períodos em que os recursos caros são ociosos ou existem ocasiões de capacidade insuficiente. Com a computação em nuvem, é possível acessar o máximo ou o mínimo necessário e dimensionar dinamicamente para atender à demanda real, pagando apenas pelo que usa.

Na AWS, servidores, bancos de dados, armazenamento e componentes de aplicativos de nível superior podem ser instanciados em segundos. É possível tratá-los como recursos temporários e descartáveis, livres da inflexibilidade e das restrições de uma infraestrutura de TI fixa e finita. Isso redefine a maneira de abordagem de gerenciamento de mudanças, testes, confiabilidade e planejamento de capacidade.

Global, disponível e capacidade ilimitada

Usando a infraestrutura global da AWS, você pode implantar seu aplicativo na região da AWS² que melhor atenda aos seus requisitos (por exemplo, proximidade com seus usuários finais, conformidade, restrições de residência de dados, custo, etc.). Para aplicativos globais, é possível reduzir a latência para usuários finais em todo o mundo usando a rede de distribuição de conteúdo do Amazon CloudFront. Também é muito mais fácil operar aplicativos de produção e bancos de dados em vários datacenters para obter alta disponibilidade e tolerância a falhas. Juntamente com a capacidade sob demanda virtualmente ilimitada que está disponível para os clientes da AWS, você pode pensar de forma diferente sobre como ativar a expansão futura por meio de sua arquitetura de TI.

Serviços gerenciados de nível superior

Além dos recursos de computação do Amazon Elastic Compute Cloud (Amazon EC2), os clientes da AWS também têm acesso a um amplo conjunto de serviços de armazenamento, banco de dados, análises, aplicativos e implantação. Como

esses serviços estão instantaneamente disponíveis para os desenvolvedores, eles reduzem a dependência de habilidades especializadas internas e permitem que as organizações forneçam novas soluções mais rapidamente. Esses serviços são gerenciados pela AWS, o que pode reduzir a complexidade operacional e o custo. Os serviços gerenciados da AWS são projetados para escalabilidade e alta disponibilidade, para que possam reduzir o risco de suas implementações.

Segurança integrada

Na TI tradicional, a auditoria de segurança de infraestrutura costuma ser um processo periódico e manual. A nuvem da AWS, em vez disso, fornece recursos de governança que permitem o monitoramento contínuo de alterações de configuração em seus recursos de TI. Como os recursos da AWS são recursos programáveis, sua política de segurança pode ser formalizada e incorporada ao design de sua infraestrutura. Com a capacidade de criar ambientes temporários, os testes de segurança agora podem se tornar parte de seu pipeline de entrega contínua. Por fim, os arquitetos de soluções podem aproveitar uma infinidade de recursos nativos de segurança e criptografia da AWS, que podem ajudar a alcançar níveis mais altos de proteção e conformidade de dados.

Princípios do design

Nesta seção, fornecemos padrões de design e opções de arquitetura que podem ser aplicados em uma ampla variedade de casos de uso.

Escalabilidade

Os sistemas que devem crescer com o tempo precisam ser construídos sobre uma arquitetura escalável. Essa arquitetura pode suportar o crescimento de usuários, tráfego ou tamanho de dados sem perda de desempenho. Ele deve fornecer essa escala de maneira linear, em que a adição de recursos extras resulta, pelo menos, em um aumento proporcional na capacidade de servir carga adicional. O crescimento deve introduzir economias de escala, e o custo deve seguir a mesma dimensão que gera valor de negócio fora desse sistema. Embora a computação em nuvem ofereça capacidade praticamente ilimitada sob demanda, seu design precisa ser capaz de aproveitar esses recursos tranquilamente. Geralmente, há duas maneiras de escalar uma arquitetura de TI: verticalmente e horizontalmente.

Escalabilidade vertical

A escalabilidade vertical ocorre através de um aumento nas especificações de um recurso individual (por exemplo, atualizando um servidor com um disco rígido maior ou uma CPU mais rápida). No Amazon EC2, isso pode ser facilmente alcançado parando uma instância e redimensionando-a para um tipo de instância que tenha mais recursos de RAM, CPU, E/S ou de rede. Essa forma de escalabilidade pode acabar atingindo um limite, o que nem sempre é uma abordagem econômica ou altamente disponível. No entanto, é muito fácil de implementar e pode ser suficiente para muitos casos de uso, especialmente a curto prazo.

Escalabilidade horizontal

A escalabilidade horizontal ocorre por meio de um aumento no número de recursos (por exemplo, adicionar mais discos rígidos a uma matriz de armazenamento ou adicionar mais servidores para suportar um aplicativo). Essa é uma ótima maneira de criar aplicativos em escala de Internet que aproveitam a elasticidade da computação em nuvem. Nem todas as arquiteturas foram projetadas para distribuir sua carga de trabalho para vários recursos, por isso, vamos examinar alguns dos cenários possíveis.

Aplicativos stateless

Quando usuários ou serviços interagem com um aplicativo, eles geralmente executam uma série de interações que formam uma sessão. Um aplicativo stateless é um aplicativo que não precisa conhecer interações anteriores e não armazena informações de sessão. Tal exemplo poderia ser um aplicativo que, dada a mesma entrada, fornece a mesma resposta para qualquer usuário final. Um aplicativo stateless pode ser dimensionado horizontalmente, pois qualquer solicitação pode ser atendida por qualquer um dos recursos de computação disponíveis (por exemplo, instâncias do EC2, funções do AWS Lambda). Sem dados de sessão a serem compartilhados, é possível simplesmente adicionar mais recursos de computação, conforme necessário. Quando essa capacidade deixa de ser necessária, qualquer recurso individual pode ser encerrado com segurança (depois de executar tarefas que foram drenadas). Esses recursos não precisam ter conhecimento da presença de seus pares, pois eles só precisam de um jeito de receber a carga de trabalho.

Como distribuir carga para vários nós

***Modelo Push:** Uma maneira popular de distribuir uma carga de trabalho é através do uso de uma solução de balanceamento de carga, como o serviço Elastic Load Balancing (ELB). O Elastic Load Balancing encaminha solicitações de aplicativos de entrada em várias instâncias do EC2. Uma abordagem alternativa seria implementar uma repetição alternada de DNS (por exemplo, com o Amazon Route 53). Nesse caso, as respostas do DNS retornam um endereço IP de uma lista de hosts válidos de maneira de repetição alternada. Embora seja fácil de implementar, essa abordagem nem sempre funciona bem com a elasticidade da computação em nuvem. Isso ocorre porque, mesmo podendo definir valores de tempo de vida (TTL) baixos para seus registros DNS, os resolvedores de DNS em cache estão fora do controle do Amazon Route 53 e, nem sempre, podem respeitar suas configurações.*

***Modelo Pull:** As cargas de trabalho orientadas por eventos assíncronos não exigem uma solução de balanceamento de carga, pois você pode implementar um modelo de recepção. Em um modelo de recepção, as tarefas que precisam ser executadas, ou os dados que precisam ser processados, podem ser armazenados como mensagens em uma fila usando o Amazon Simple Queue Service (Amazon SQS) ou como uma solução de dados de streaming como o Amazon Kinesis. Então, vários nós de computação podem puxar e consumir essas mensagens, processando-as de maneira distribuída.*

Componentes stateless

Na prática, a maioria dos aplicativos precisa manter algum tipo de informação de estado. Por exemplo, os aplicativos web precisam rastrear se um usuário está conectado ou podem apresentar conteúdo personalizado com base em ações anteriores. Um processo automatizado de várias etapas também precisará rastrear atividades anteriores para decidir qual deve ser a próxima ação. É possível tornar uma parte dessas arquiteturas stateless, não armazenando nada no sistema de arquivos local que precise persistir por mais de uma única solicitação.

Por exemplo, os aplicativos web podem usar cookies HTTP para armazenar informações sobre uma sessão no navegador do cliente (por exemplo, itens no carrinho de compras). O navegador repassa essas informações para o servidor em cada solicitação subsequente para que o aplicativo não precise armazená-las. No entanto, existem duas desvantagens com essa abordagem. Primeiro, o conteúdo dos cookies HTTP pode ser adulterado no lado do cliente, portanto, você deve sempre tratá-los como dados não confiáveis que precisam ser validados. Em segundo lugar, os cookies HTTP são transmitidos a cada solicitação, o que

significa que você deve manter seu tamanho ao mínimo (para evitar latência desnecessária).

Considere armazenar apenas um identificador de sessão exclusivo em um cookie HTTP e armazenar informações mais detalhadas da sessão do usuário no lado do servidor. A maioria das plataformas de programação fornece um mecanismo de gerenciamento de sessão nativo que funciona dessa maneira, no entanto, isso geralmente é armazenado no sistema de arquivos local por padrão. Isso resultaria em uma arquitetura stateful. Uma solução comum para esse problema é armazenar informações de sessão do usuário em um banco de dados. O Amazon DynamoDB é uma ótima opção devido às suas características de escalabilidade, alta disponibilidade e durabilidade. Para muitas plataformas, há bibliotecas de substituição direta de software livre que permitem armazenar sessões nativas no Amazon DynamoDB.³

Outros cenários exigem armazenamento de arquivos maiores (por exemplo, uploads de usuários, resultados provisórios de processos em lote, etc.). Ao colocar esses arquivos em uma camada de armazenamento compartilhado, como o Amazon S3 ou o Amazon Elastic File System (Amazon EFS), você pode evitar a introdução de componentes stateful. Outro exemplo é o de um fluxo de trabalho complexo de várias etapas, no qual você precisa acompanhar o estado atual de cada execução. O Amazon Simple Workflow Service (Amazon SWF) pode ser utilizado para armazenar centralmente o histórico de execução e tornar essas cargas de trabalho stateless.

Componentes stateful

Inevitavelmente, haverá camadas de sua arquitetura que você não transformará em componentes stateless. Primeiro, por definição, os bancos de dados são stateful. (Eles serão abordados separadamente na seção de Bancos de dados .) Além disso, muitos aplicativos legados foram projetados para serem executados em um único servidor, contando com recursos de computação locais. Outros casos de uso podem exigir que os dispositivos clientes mantenham uma conexão com um servidor específico por períodos prolongados. Por exemplo, os jogos multijogador em tempo real devem oferecer a vários jogadores uma visão consistente do mundo do jogo com latência muito baixa. Isso é muito mais simples de se conseguir em uma implementação não distribuída na qual os participantes estão conectados ao mesmo servidor.

Você ainda pode escalar esses componentes horizontalmente, distribuindo a carga em vários nós com “afinidade de sessão”. Neste modelo, você associa todas as transações de uma sessão a um recurso de computação específico. Você deve estar ciente das limitações deste modelo. As sessões existentes não se beneficiam diretamente da introdução de nós de computação recém-lançados. E o mais importante é que a afinidade de sessão não pode ser garantida. Por exemplo, quando um nó é finalizado ou fica indisponível, os usuários associados a ele serão desconectados e terão uma perda de dados específicos da sessão (por exemplo, qualquer coisa que não esteja armazenada em um recurso compartilhado como S3, EFS, um banco de dados, etc.).

Como implementar afinidade de sessão

Para tráfego HTTP/S, a afinidade de sessão pode ser obtida através do recurso “sticky sessions” do ELB.⁴ O Elastic Load Balancing tentará usar o mesmo servidor para esse usuário pela duração da sessão.

Outra opção é usar o balanceamento de carga do lado do cliente, se você controlar o código executado no cliente. Isso adiciona complexidade extra, mas pode ser útil em cenários em que um load balancer não atenda aos seus requisitos. Por exemplo, você pode estar usando um protocolo não suportado pelo ELB ou pode precisar de controle total sobre como os usuários são atribuídos a servidores (por exemplo, em um cenário de jogo, talvez seja necessário verificar se os participantes do jogo são correspondidos e se conectam ao mesmo servidor). Neste modelo, os clientes precisam de uma maneira de descobrir os endpoints do servidor válidos para se conectarem diretamente. Você pode usar o DNS para isso ou pode criar uma API de descoberta simples para fornecer essas informações ao software em execução no cliente. Na ausência de um load balancer, o mecanismo de verificação de integridade também precisará ser implementado no lado do cliente. Você deve projetar sua lógica de cliente para que, quando a indisponibilidade do servidor for detectada, os dispositivos possam se reconectar a outro servidor com pouca interrupção para o aplicativo.

Processamento distribuído

Os casos de uso que envolvem o processamento de grandes quantidades de dados (por exemplo, tudo o que não pode ser tratado por um único recurso de computação em tempo hábil) exigem uma abordagem de processamento distribuído. Ao dividir uma tarefa e seus dados em vários fragmentos pequenos de trabalho, você pode executar cada um deles em qualquer um dos maiores conjuntos de recursos de computação disponíveis.

Como implementar o processamento distribuído

Tarefas em lote off-line podem ser dimensionadas horizontalmente usando um mecanismo de processamento de dados distribuído como o Apache Hadoop. Na AWS, é possível usar o serviço Amazon Elastic MapReduce (Amazon EMR) para executar cargas de trabalho do Hadoop sobre uma frota de instâncias do EC2 sem a complexidade operacional. Para processamento em tempo real de dados de streaming, o Amazon Kinesis particiona dados em vários fragmentos que podem ser consumidos por vários recursos do Amazon EC2 ou AWS Lambda para obter escalabilidade.

Para obter mais informações sobre esses tipos de cargas de trabalho, consulte o whitepaper sobre “Opções de análise de big data na AWS”⁵.

Recursos descartáveis em vez de servidores fixos

Em um ambiente de infraestrutura tradicional, você precisa trabalhar com recursos fixos devido ao custo inicial e ao prazo de introdução de novo hardware. Isso levaria práticas como fazer login manualmente em servidores, a configurar software ou corrigir problemas, codificar endereços IP, executar testes ou processar trabalhos sequencialmente, etc.

Ao projetar para a AWS, você tem a oportunidade de redefinir essa mentalidade para aproveitar a natureza provisionada dinamicamente da computação em nuvem. Você pode pensar em servidores e outros componentes como recursos temporários. Você pode lançar quantos precisar e usá-los apenas pelo tempo que precisar.

Outro problema com servidores fixos e de longa duração é o de *desvio de configuração*. Alterações e patches de software aplicados ao longo do tempo podem resultar em configurações não testadas e heterogêneas em diferentes ambientes. Esse problema pode ser resolvido com o padrão de *infraestrutura imutável*. Com essa abordagem, um servidor iniciado, nunca é atualizado ao longo de sua vida útil. Em vez disso, quando há um problema ou uma necessidade de atualização, o servidor é substituído por um novo que tenha a configuração mais recente. Dessa maneira, os recursos estão sempre em um estado consistente (e testado), e as reversões se tornam mais fáceis de executar.

Instanciando recursos de computação

Se estiver implantando um novo ambiente para teste ou aumentando a capacidade de um sistema existente para lidar com carga extra, você não desejará configurar manualmente novos recursos com sua configuração e código. É importante que você faça disso um processo automatizado e repetitivo que evita longos tempos de espera e não é propenso a erros humanos.

Existem algumas abordagens sobre como obter um processo automatizado e repetitivo.

Bootstrapping

Quando iniciar um recurso da AWS como uma instância do Amazon EC2 ou uma instância de banco de dados do banco de dados relacional da Amazon (Amazon RDS), você começa com uma configuração padrão. Assim, você pode executar ações automatizadas de bootstrapping. Ou seja, scripts que instalam software ou copiam dados para levar esse recurso a um estado específico. Você pode parametrizar detalhes de configuração que variam entre diferentes ambientes (por exemplo, produção, teste, etc.), para que os mesmos scripts possam ser reutilizados sem modificações.

Bootstrapping na prática

Você pode usar scripts de dados do usuário e cloud-init⁶ diretivas ou eventos do ciclo de vida do AWS OpsWorks⁷ para configurar automaticamente novas instâncias do EC2. É possível usar scripts simples, ferramentas de gerenciamento de configuração como Chef ou Puppet. O AWS OpsWorks oferece suporte nativamente a receitas do Chef ou scripts Bash/PowerShell. Além disso, por meio de scripts personalizados e APIs da AWS, ou pelo uso do suporte do AWS CloudFormation para recursos personalizados com suporte do AWS Lambda⁸, é possível escrever uma lógica de provisionamento que atue em quase todos os recursos da AWS.

Imagens douradas

Determinados tipos de recursos da AWS, como instâncias do Amazon EC2, instâncias do Amazon RDS DB, volumes do Amazon Elastic Block Store (Amazon EBS), etc., podem ser executados a partir de uma imagem dourada: um snapshot de um estado específico desse recurso. Quando comparado com a abordagem de bootstrapping, uma imagem dourada resulta em períodos de início mais rápidos, e remove dependências para serviços de configuração ou repositórios de terceiros. Isso é importante em ambientes com escala

automática em que você deseja lançar recursos adicionais de maneira rápida e confiável, como uma resposta às alterações na demanda.

Você pode personalizar uma instância do Amazon EC2 e salvar sua configuração criando uma Amazon Machine Image (AMI)⁹. Você pode iniciar quantas instâncias da AMI forem necessárias e todas elas incluirão as personalizações feitas por você. Cada vez que quiser alterar sua configuração, será necessário criar uma nova imagem dourada. Portanto, você precisará ter uma convenção de controle de versão para gerenciar suas imagens douradas ao longo do tempo. Recomendamos que use um script para inicializar as instâncias do EC2 usadas para criar suas AMIs. Isso dará uma maneira flexível de testar e modificar essas imagens ao longo do tempo.

Como alternativa, se tiver um ambiente virtualizado no local já existente, você poderá usar o VM Import/Export da AWS para converter uma variedade de formatos de virtualização em uma AMI. Você também pode encontrar e usar AMIs compartilhadas pré-preenchidas, fornecidas pela AWS ou por terceiros no catálogo da AMI da AWS Community, ou no AWS Marketplace.

Embora as imagens douradas sejam mais usadas ao iniciar novas instâncias do EC2, elas também podem ser aplicadas a recursos como bancos de dados do Amazon RDS ou volumes do Amazon EBS. Por exemplo, ao iniciar um novo ambiente de teste, você pode preencher previamente seu banco de dados, instanciando-o a partir de um snapshot específico do Amazon RDS, em vez de importar os dados de um longo script SQL.

Contêineres

Outra opção popular entre os desenvolvedores é o Docker - uma tecnologia de código aberto que permite construir e implantar aplicativos distribuídos dentro de contêineres de software. O Docker permite integrar um software em uma Docker Image, que é uma unidade padronizada para desenvolvimento de software, contendo tudo o que o software precisa para execução: código, tempo de execução, ferramentas do sistema, bibliotecas do sistema, etc. AWS Elastic Beanstalk e o Amazon EC2 Container Service (Amazon ECS) suporta o Docker e permite que você implante e gerencie vários contêineres do Docker em um cluster de instâncias do Amazon EC2.

Híbrido

É possível usar uma combinação das duas abordagens, em que algumas partes da configuração são capturadas em uma imagem dourada, enquanto outras são configuradas dinamicamente por meio de uma ação de bootstrapping.

A linha entre bootstrapping e a imagem dourada

Itens que não mudam com frequência ou que introduzem dependências externas normalmente fazem parte de sua imagem dourada. Por exemplo, seu software de servidor web que teria de ser baixado por um repositório de terceiros toda vez que você iniciasse uma instância é uma boa opção.

Itens que mudam frequentemente ou diferem entre seus vários ambientes podem ser configurados dinamicamente através de ações de bootstrapping. Por exemplo, se estiver implantando novas versões de seu aplicativo com frequência, criar uma nova AMI para cada versão do aplicativo pode ser impraticável. Você também não desejaria codificar a configuração do nome do host do banco de dados em sua AMI porque isso seria diferente entre os ambientes de teste e de produção. Os dados ou tags do usuário podem ser usados para permitir que você use AMIs mais genéricas que podem ser modificadas no momento do lançamento. Por exemplo, se executar servidores web para várias empresas de pequeno porte, todos eles poderão usar a mesma AMI e recuperar seu conteúdo de um local de bucket do Amazon S3 especificado nos dados do usuário na inicialização.

O AWS Elastic Beanstalk segue o modelo híbrido. Ele fornece ambientes de tempo de execução pré-configurados (cada um iniciado a partir de sua própria AMI¹⁰), mas permite executar ações de bootstrap (através de arquivos de configuração chamados .ebextensions¹¹) e configurar variáveis ambientais para parametrizar as diferenças de ambiente.

Para obter uma discussão mais detalhada das diferentes maneiras de gerenciar implantações de novos recursos, consulte os whitepapers [Visão geral das opções de implantação na AWS](#) e [Gerenciando sua infraestrutura da AWS em escala](#).

Infraestrutura como código

A aplicação dos princípios que discutimos não precisa ser limitada ao nível de recursos individuais. Como os recursos da AWS são programáveis, você pode aplicar técnicas, práticas e ferramentas do desenvolvimento de software para tornar toda a sua infraestrutura reutilizável, sustentável, extensível e testável.

*Os modelos do **AWS CloudFormation** fornecem aos desenvolvedores e administradores de sistemas uma maneira fácil de criar e gerenciar uma coleção de recursos da AWS relacionados, além de provisioná-los e atualizá-los de maneira ordenada e previsível. Você pode descrever os recursos da AWS e quaisquer dependências associadas ou parâmetros de tempo de execução necessários para executar seu aplicativo. Seus modelos do CloudFormation podem viver com o aplicativo em seu repositório de controle de versão, permitindo que as arquiteturas sejam reutilizadas e que os ambientes de produção sejam clonados de forma confiável para testes.*

Automation

Em uma infraestrutura de TI tradicional, você teria que reagir manualmente a vários eventos. Ao implantar na AWS, há muitas oportunidades de automação para melhorar a estabilidade do sistema e a eficiência da sua organização:

- **AWS Elastic Beanstalk**¹² é a maneira mais rápida e simples de colocar um aplicativo em funcionamento na AWS. Os desenvolvedores podem simplesmente fazer o upload de seu código de aplicativo, e o serviço administra automaticamente todos os detalhes, como provisionamento de recursos, balanceamento de carga, Auto Scaling e monitoramento.
- **Recuperação automática do Amazon EC2**¹³: Você pode criar um alarme do Amazon CloudWatch que monitore uma instância do Amazon EC2 e a recupere automaticamente caso ela fique prejudicada. Uma instância recuperada é idêntica à instância original, incluindo o ID da instância, os endereços IP privados, os endereços IP elásticos e todos os metadados da instância. No entanto, esse recurso só está disponível para configurações de instância aplicáveis. Consulte a documentação do Amazon EC2 para obter uma descrição atualizada dessas condições prévias. Além disso, durante a recuperação da instância, a mesma é migrada por meio de uma reinicialização de instância e todos os dados que estão na memória são perdidos.
- **Auto Scaling**¹⁴: Com o Auto Scaling, é possível manter a disponibilidade de aplicativos e dimensionar a capacidade do Amazon EC2 automaticamente de acordo com as condições definidas por você. Você pode usar o Auto Scaling para ajudar a garantir que esteja executando o número desejado de instâncias saudáveis do Amazon EC2 em várias Zonas de disponibilidade. O Auto Scaling também pode aumentar automaticamente o número de instâncias do Amazon EC2 durante picos de demanda para manter o desempenho e diminuir a capacidade durante períodos menos movimentados para otimizar os custos.

- **Alarmes do Amazon CloudWatch¹⁵:** Você pode criar um alarme do CloudWatch que envie uma mensagem do Amazon Simple Notification Service (Amazon SNS) quando uma determinada métrica ultrapassar um limite especificado por um número específico de períodos. Essas mensagens do Amazon SNS podem iniciar automaticamente a execução de uma função do AWS Lambda inscrita, enfileirar uma mensagem de notificação em uma fila do Amazon SQS ou executar uma solicitação POST em um endpoint HTTP/S.
- **Amazon CloudWatch Events¹⁶:** O serviço CloudWatch oferece um fluxo quase em tempo real de eventos do sistema que descrevem as alterações nos recursos da AWS. Usando regras simples que podem ser configuradas em poucos minutos, é possível rotear cada tipo de evento para um ou mais destinos: Funções do AWS Lambda, fluxos do Amazon Kinesis, tópicos do Amazon SNS, etc.
- **Eventos do ciclo de vida do AWS OpsWorks¹⁷:** O AWS OpsWorks suporta configuração contínua por meio de eventos de ciclo de vida que atualizam automaticamente a configuração de suas instâncias para se adaptar às alterações do ambiente. Esses eventos podem ser usados para acionar receitas do Chef em cada instância a fim de executar tarefas de configuração específicas. Por exemplo, quando uma nova instância é adicionada com sucesso a uma camada do servidor de banco de dados, o evento configurar poderia acionar uma receita do Chef que atualiza a configuração da camada do servidor de aplicativos para apontar para a nova instância do banco de dados.
- **Eventos programados do AWS Lambda¹⁸:** Esses eventos permitem criar uma função do Lambda e direcionar o AWS Lambda para executá-lo regularmente.

Baixo acoplamento

À medida que a complexidade do aplicativo aumenta, um atributo desejável de um sistema de TI é que ele pode ser dividido em componentes menores e frouxamente acoplados. Isso significa que os sistemas de TI devem ser projetados de uma maneira que reduza as interdependências. Uma alteração ou uma falha em um componente não deve ocorrer em cascata para outros componentes.

Interfaces bem definidas

Uma forma de reduzir as interdependências em um sistema é permitir que os vários componentes interajam entre si somente por meio de interfaces específicas, independentes de tecnologia (por exemplo, APIs RESTful). Dessa

forma, os detalhes da implementação técnica ficam ocultos para que as equipes possam modificar a implementação subjacente sem afetar outros componentes. Desde que essas interfaces mantenham a compatibilidade retroativa, as implantações de componentes de diferença são desacopladas.

*O **Amazon API Gateway** é um serviço totalmente gerenciado que permite que desenvolvedores criem, publiquem, mantenham, monitorem e protejam APIs em qualquer escala com facilidade. Ele administra todas as tarefas envolvidas na aceitação e no processamento de centenas de milhares de chamadas de API simultâneas, incluindo gerenciamento de tráfego, autorização e controle de acesso, monitoramento e gerenciamento de versão da API.*

Descoberta de serviço

Os aplicativos implantados como um conjunto de serviços menores dependerão da capacidade desses serviços de interagirem entre si. Como cada um desses serviços pode estar sendo executado em vários recursos de computação, é necessário que haja uma maneira de abordar cada serviço. Por exemplo, em uma infraestrutura tradicional, se o serviço web de front-end precisasse se conectar ao serviço web de back-end, você poderia codificar o endereço IP do recurso de computação no qual esse serviço estava sendo executado. Embora essa abordagem ainda possa funcionar na computação em nuvem, se esses serviços forem destinados a serem frouxamente acoplados, eles devem poder ser consumidos sem o conhecimento prévio de seus detalhes de topologia de rede. Além de ocultar a complexidade, isso também permite que os detalhes da infraestrutura sejam alterados a qualquer momento. O baixo acoplamento é um elemento crucial se quiser aproveitar a elasticidade da computação em nuvem, na qual novos recursos podem ser iniciados ou terminados a qualquer momento. Para conseguir isso, você precisará de alguma maneira implementar a descoberta de serviços.

Como implementar a descoberta de serviço

Para um serviço hospedado no Amazon EC2, uma maneira simples de obter descoberta de serviço é por meio do serviço Elastic Load Balancing. Como cada load balancer tem seu próprio nome de host, agora você pode consumir um serviço por meio de um endpoint estável. Isso pode ser combinado com DNS e zonas privadas do Amazon Route53, para que até mesmo o endpoint do load balancer específico possa ser abstraído e modificado a qualquer momento.

Outra opção seria usar um método de registro e descoberta de serviço para permitir a recuperação dos endereços IP e do número de porta do endpoint de qualquer serviço determinado. Como a descoberta de serviços se torna o vínculo entre os componentes, é importante que seja altamente disponível e confiável. Se os load balancers não forem usados, a descoberta de serviços também deve servir para coisas como verificação de integridade. Exemplos de implementações incluem soluções personalizadas usando uma combinação de tags, um banco de dados altamente disponível e scripts personalizados que chamam as APIs da AWS, ou ferramentas de código aberto como Netflix Eureka, Airbnb Synapse ou HashiCorp Consul.

Integração assíncrona

A integração assíncrona é outra forma de baixo acoplamento entre serviços. Esse modelo é adequado para qualquer interação que não precise de uma resposta imediata e onde um reconhecimento de que uma solicitação foi registrada será suficiente. Envolve um componente que gera eventos e outro que os consome. Os dois componentes não se integram por meio da interação direta ponto-a-ponto, mas geralmente por meio de uma camada intermediária de armazenamento durável (por exemplo, uma fila do Amazon SQS ou uma plataforma de dados de streaming como o Amazon Kinesis).

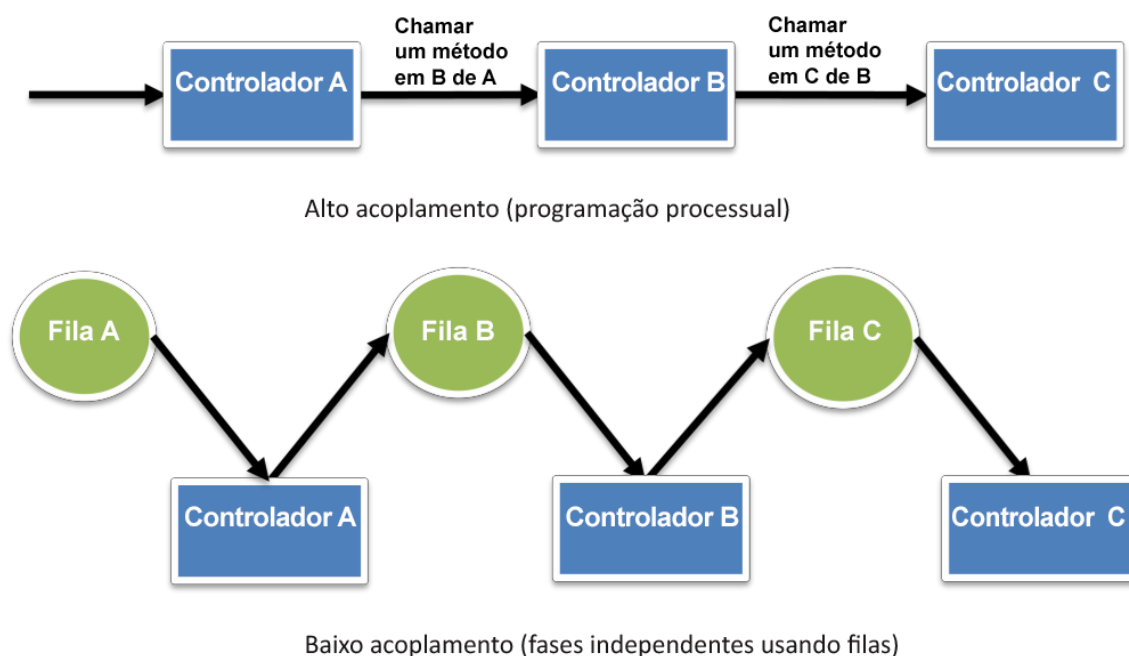


Figura 1: Alto e baixo acoplamento

Essa abordagem separa os dois componentes e apresenta resiliência adicional. Então, por exemplo, se um processo que estiver lendo mensagens da fila falhar, as mensagens ainda poderão ser adicionadas à fila para serem processadas quando o sistema for recuperado. Ele também permite que você proteja um serviço de back-end menos escalável dos picos de front-end, e encontre a relação correta entre custo e atraso de processamento. Por exemplo, você pode decidir que não precisa dimensionar seu banco de dados para acomodar um pico ocasional de consultas de gravação, desde que processe essas consultas de forma assíncrona com algum atraso. Finalmente, ao retirar as operações lentas dos caminhos de solicitações interativas, você também pode melhorar a experiência do usuário final.

Exemplos de integração assíncrona

- Um aplicativo front-end insere trabalhos em um sistema de filas como o Amazon SQS. Um sistema back-end recupera essas tarefas e as processa em seu próprio ritmo.
- Uma API gera eventos e os envia para os fluxos do Amazon Kinesis. Um aplicativo de back-end processa esses eventos em lotes para criar dados de séries temporais agregados e armazenados em um banco de dados.
- Vários sistemas heterogêneos usam o Amazon SWF para comunicar o fluxo de trabalho entre eles, sem interagir diretamente uns com os outros.
- As funções do AWS Lambda podem consumir eventos de diversas fontes da AWS (por exemplo, fluxos de atualização do Amazon DynamoDB, notificações de eventos do Amazon S3, etc.). Nesse caso, você não precisa se preocupar com a implementação de um enfileiramento ou outro método de integração assíncrona porque o serviço administra isso para você.

Falha normal

Outra maneira de aumentar o baixo acoplamento é criar aplicativos de maneira que eles lidem com falhas de componentes de maneira normal. Você pode identificar maneiras de reduzir o impacto para os usuários finais e aumentar sua capacidade de progredir em seus procedimentos off-line, mesmo em caso de falha de algum componente.

Falha normal na prática

Uma solicitação que falha pode ser repetida com uma estratégia de recuo exponencial e tremulação, ou pode ser armazenada em uma fila para processamento posterior.¹⁹ Para interfaces front-end, pode ser possível fornecer conteúdo alternativo ou em cache, em vez de falhar completamente quando, por exemplo, o servidor de banco de dados ficar indisponível. O recurso de failover de DNS do Amazon Route 53 também oferece a capacidade de monitorar seu site e rotear automaticamente seus visitantes para um site de backup se o site principal ficar indisponível. Você pode hospedar seu site de backup como um site estático no Amazon S3 ou como um ambiente dinâmico separado.

Serviços, não servidores

O desenvolvimento, o gerenciamento e a operação de aplicativos, especialmente em escala, exigem uma ampla variedade de componentes tecnológicos subjacentes. Com a infraestrutura de TI tradicional, as empresas teriam que construir e operar todos esses componentes.

A AWS oferece um amplo conjunto de serviços de computação, armazenamento, banco de dados, análises, aplicativos e implantação que ajudam as organizações a se movimentarem mais rapidamente e a reduzir os custos de TI. Arquiteturas que não aproveitam essa amplitude (por exemplo, se usarem apenas o Amazon EC2) talvez não aproveitem ao máximo a computação em nuvem e podem perder uma oportunidade de aumentar a produtividade e a eficiência operacional do desenvolvedor.

Serviços gerenciados

Na AWS, há um conjunto de serviços que fornecem blocos de construção que os desenvolvedores podem consumir para alimentar seus aplicativos. Esses serviços gerenciados incluem bancos de dados, machine learning, análise, enfileiramento, pesquisa, e-mail, notificações e muito mais. Por exemplo, com o Amazon Simple Queue Service (Amazon SQS), você pode aliviar a carga administrativa de operar e dimensionar um cluster de mensagens altamente disponível, pagando um preço baixo apenas pelo que você usa. Além disso, o Amazon SQS é inerentemente escalável. O mesmo se aplica ao Amazon S3, onde você pode armazenar quantos dados desejar e acessá-los quando necessário, sem precisar pensar em capacidade, configurações de disco rígido, replicação etc. Além disso, o Amazon S3 também pode veicular ativos estáticos de um aplicativo da Web ou móvel, fornecendo uma solução de hospedagem altamente disponível que pode ser dimensionada automaticamente para atender às demandas de tráfego.

Há muitos outros exemplos, como Amazon CloudFront para entrega de conteúdo, ELB para balanceamento de carga, Amazon DynamoDB para bancos de dados NoSQL, Amazon CloudSearch para cargas de trabalho de pesquisa, Amazon Elastic Transcoder para codificação de vídeo, Amazon Simple Email Service (Amazon SES) para envio e recebimento de e-mails, e outros.²⁰

Arquiteturas sem servidor

Outra abordagem que pode reduzir a complexidade operacional dos aplicativos em execução é a das arquiteturas sem servidor. É possível construir serviços orientados a eventos e síncronos para dispositivos móveis, web, analíticos e a Internet das coisas (IoT) sem gerenciar qualquer infraestrutura de servidor. Essas arquiteturas podem reduzir custos porque você não está pagando por servidores subutilizados, nem provisionando infraestrutura redundante para implementar alta disponibilidade.

*É possível enviar seu código para o serviço de computação **AWS Lambda** e o serviço pode executar o código em seu nome usando a infraestrutura da AWS. Com o AWS Lambda, você é cobrado por cada 100 ms executado por código e pelo número de vezes que seu código é acionado. Usando o Amazon API Gateway, você pode desenvolver APIs síncronas virtualmente infinitamente escaláveis, desenvolvido pelo AWS Lambda. Quando combinado com o Amazon S3 para servir ativos de conteúdo estático, esse padrão pode entregar um aplicativo web completo. Para obter mais detalhes sobre este tipo de arquitetura, consulte o whitepaper “Arquiteturas de múltiplas camadas sem servidor da AWS”.²¹*

Quando se trata de aplicativos móveis, há mais uma maneira de reduzir a superfície de uma infraestrutura baseada em servidor. Você pode utilizar o Amazon Cognito, assim não precisa gerenciar uma solução de back-end para lidar com a autenticação do usuário, o estado da rede, o armazenamento e a sincronização. O Amazon Cognito gera identificadores exclusivos para seus usuários. Esses podem ser referenciadas em suas políticas de acesso para permitir ou restringir o acesso a outros recursos da AWS por usuário. O Amazon Cognito fornece credenciais temporárias da AWS a seus usuários, permitindo que o aplicativo móvel em execução no dispositivo interaja diretamente com serviços da AWS protegidos pelo AWS Identity and Access Management (IAM). Por exemplo, usando o IAM, você pode restringir o acesso a uma pasta em um bucket do Amazon S3 para um determinado usuário final.

Para aplicativos IoT, tradicionalmente as organizações tiveram que provisionar, operar, dimensionar e manter seus próprios servidores como gateways de dispositivos para lidar com a comunicação entre os dispositivos conectados e seus serviços. O AWS IoT fornece um gateway de dispositivo totalmente gerenciado que é dimensionado automaticamente com o seu uso, sem qualquer sobrecarga operacional para você.

Bancos de dados

Com a infraestrutura de TI tradicional, as organizações costumavam a se limitar às tecnologias de banco de dados e armazenamento que podiam usar. Pode haver restrições com base nos custos de licenciamento e na capacidade de suportar diversos mecanismos de banco de dados. Na AWS, essas restrições são removidas pelos serviços de banco de dados gerenciados que oferecem desempenho corporativo a custo de código aberto. Como resultado, não é incomum que os aplicativos sejam executados sobre uma camada de dados poliglota, escolhendo a tecnologia certa para cada carga de trabalho.

Determinando a tecnologia de banco de dados certa para cada carga de trabalho

As perguntas a seguir podem ajudá-lo a tomar decisões sobre quais soluções incluir em sua arquitetura:

- *Esta é uma carga de trabalho de leitura intensa, gravação intensa ou balanceada? De quantas leituras e gravações por segundo você vai precisar? Como esses valores mudarão se o número de usuários aumentar?*
- *Quantos dados você precisará armazenar e por quanto tempo? Qual a velocidade prevista de crescimento? Existe um limite superior no futuro previsível? Qual é o tamanho de cada objeto (média, mínimo, máximo)? Como esses objetos serão acessados?*
- *Quais são os requisitos em termos de durabilidade dos dados? Essa loja de dados será sua “fonte da verdade”?*
- *Quais são os seus requisitos de latência? Para quantos usuários simultâneos você precisa dar suporte?*
- *Qual é o seu modelo de dados e como você vai consultar esses dados? Suas consultas são de natureza relacional (por exemplo, JOINS entre várias tabelas)? Você poderia desnormalizar seu esquema para criar estruturas de dados mais planas e mais fáceis de dimensionar?*
- *Qual tipo de funcionalidade você precisa? Você precisa de controles de integridade fortes ou está procurando mais flexibilidade (por exemplo, armazenamentos de dados sem esquema)? Você precisa de relatórios sofisticados ou recursos de pesquisa? Seus desenvolvedores estão mais familiarizados com bancos de dados relacionais do que o NoSQL?*

Esta seção discute as diferentes categorias de tecnologias de banco de dados a serem consideradas.

Bancos de dados relacionais

Bancos de dados relacionais (geralmente chamados bancos de dados RDBS ou SQL) normalizam os dados em estruturas tabulares bem definidas, conhecidas como tabelas, que consistem de linhas e colunas. Eles fornecem uma poderosa linguagem de consulta, recursos de indexação flexíveis, controles de integridade fortes e a capacidade de combinar dados de várias tabelas de maneira rápida e eficiente. O Amazon Relational Database Service (Amazon RDS) facilita configurar, operar e ajustar a escala de bancos de dados relacionais na nuvem.

Escalabilidade

Os bancos de dados relacionais podem ser dimensionados verticalmente (por exemplo, atualizando para uma instância DB maior do Amazon RDS ou adicionando armazenamento maior e mais rápido). Além disso, considere o uso do Amazon RDS for Aurora, que é um mecanismo de banco de dados projetado para fornecer um rendimento muito mais alto em comparação com o padrão do MySQL em execução no mesmo hardware. Para aplicativos de leitura intensa, você também pode dimensionar horizontalmente além das restrições de capacidade de uma única instância de banco de dados criando uma ou mais réplicas de leitura.

Como aproveitar as réplicas de leitura

Réplicas de leitura são instâncias de banco de dados separadas que são replicadas de forma assíncrona. Como resultado, elas estão sujeitas a atrasos de replicação e podem estar faltando algumas das transações mais recentes. Os designers de aplicativos precisam considerar quais consultas têm tolerância a dados ligeiramente obsoletos. Essas consultas podem ser executadas em uma réplica de leitura, enquanto o restante deve ser executado no nó primário. As réplicas de leitura também não podem aceitar consultas de gravação.

Cargas de trabalho de banco de dados relacional que precisam escalar sua capacidade de gravação além das restrições de uma única instância de banco de dados exigem uma abordagem diferente chamada *particionamento de dados* ou *fragmentação*. Com esse modelo, os dados são divididos em vários esquemas de bancos de dados, cada um executando em sua própria instância de banco de dados principal autônoma. Embora o Amazon RDS elimine a sobrecarga

operacional de executar essas instâncias, a fragmentação apresenta alguma complexidade ao aplicativo. A camada de acesso a dados do aplicativo precisará ser modificada para ter consciência de como os dados são divididos, de modo que possam direcionar as consultas para a instância correta. Além disso, quaisquer alterações de esquema terão que ser executadas em vários esquemas de banco de dados, portanto, vale a pena se esforçar para automatizar esse processo.

Alta disponibilidade

Para qualquer banco de dados relacional de produção, recomendamos o uso do recurso de implantação Multi-AZ do Amazon RDS, que cria uma instância em espera replicada, em sincronia, em uma Zona de disponibilidade (AZ) diferente. Em caso de falha do nó primário, o Amazon RDS realiza um failover automático para o modo de espera sem a necessidade de intervenção administrativa manual. Quando um failover é executado, há um período curto durante o qual o nó primário não está acessível. Aplicativos resilientes podem ser projetados para Falha normal oferecendo funcionalidade reduzida (por exemplo, modo somente leitura, utilizando réplicas de leitura).

Antipadrões

Se seu aplicativo indexar e consultar dados principalmente sem necessidade de junções ou transações complexas (especialmente se você espera uma taxa de transferência de gravação além das restrições de uma única instância), considere um banco de dados NoSQL. Se você tiver arquivos binários grandes (áudio, vídeo e imagem), será mais eficiente armazenar os arquivos reais no Amazon Simple Storage Service (Amazon S3) e manter apenas os metadados dos arquivos em seu banco de dados.

Para obter informações sobre as melhores práticas de banco de dados relacional mais detalhadas, consulte a documentação do Amazon RDS.²²

Bancos de dados NoSQL

NoSQL é um termo usado para descrever bancos de dados que comercializam alguns dos recursos de consulta e transação de bancos de dados relacionais para um modelo de dados mais flexível que se equilibra horizontalmente. Os bancos de dados NoSQL utilizam uma variedade de modelos de dados, incluindo gráficos, pares de valor-chave e documentos JSON. Os bancos de dados NoSQL são amplamente reconhecidos pela facilidade de desenvolvimento, desempenho

escalável, alta disponibilidade e resiliência. O Amazon DynamoDB é um serviço de banco de dados NoSQL rápido e flexível²³ para aplicativos que precisam de latência de milissegundos consistente de um dígito em qualquer escala. É um banco de dados em nuvem totalmente gerenciado e suporta modelos de armazenamento de documentos e valor-chave.

Escalabilidade

Os mecanismos de banco de dados NoSQL normalmente executam particionamento e replicação de dados para dimensionar as leituras e as gravações de maneira horizontal. Eles fazem isso de maneira transparente, sem a necessidade de ter a lógica de particionamento de dados implementada na camada de acesso a dados de seu aplicativo. O Amazon DynamoDB, em particular, gerencia o particionamento de tabela automaticamente, adicionando novas partições à medida que sua tabela aumenta de tamanho, ou como alterações de capacidade provisionadas por leitura e gravação.

Para aproveitar ao máximo a escalabilidade do Amazon DynamoDB ao projetar seu aplicativo, consulte a seção de Melhores práticas do Amazon DynamoDB²⁴ da documentação.

Alta disponibilidade

O serviço Amazon DynamoDB replica dados de forma síncrona em três instalações em uma região da AWS para fornecer tolerância a falhas no caso de uma falha do servidor ou interrupção da Zona de disponibilidade.

Antipadrões

Se seu esquema não puder ser desnormalizado e seu aplicativo exigir junções ou transações complexas, considere um banco de dados relacional. Se você tiver arquivos binários grandes (áudio, vídeo e imagem), considere armazenar os arquivos no Amazon S3 e armazenar os metadados dos arquivos em seu banco de dados.

Ao migrar ou avaliar quais cargas de trabalho migrar de um banco de dados relacional para o DynamoDB, você pode consultar o whitepaper “Melhores práticas para migrar do RDBMS para o DynamoDB”²⁵ para mais orientação.

Data warehouse

Um data warehouse é um tipo especializado de banco de dados relacional, otimizado para análise e geração de relatórios de grandes quantidades de dados. Ele pode ser usado para combinar dados transacionais de fontes diferentes (por exemplo, comportamento do usuário em um aplicativo web, dados de seu sistema financeiro e de faturamento, CRM, etc.), tornando-os disponíveis para análise e tomada de decisões.

Tradicionalmente, configurar, executar e dimensionar um data warehouse tem sido complicado e caro. Na AWS, você pode utilizar o Amazon Redshift, um serviço de data warehouse gerenciado projetado para operar a menos de um décimo do custo de soluções tradicionais.

Escalabilidade

O Amazon Redshift obtém armazenamento eficiente e desempenho de consulta ideal por meio de uma combinação de processamento massivamente paralelo (MPP), armazenamento de dados colunares e esquemas de codificação de compactação de dados direcionados. É particularmente adequado para cargas de trabalho analíticas e de relatórios em relação a conjuntos de dados muito grandes. A arquitetura do Amazon Redshift MPP permite aumentar o desempenho, aumentando o número de nós em seu cluster do data warehouse.

Alta disponibilidade

O Amazon Redshift tem vários recursos que aprimoram a confiabilidade de seu cluster de data warehouse. Recomendamos implantar cargas de trabalho de produção em clusters de vários nós nos quais os dados gravados em cada nó são automaticamente replicados para outros nós no cluster. Os dados também são continuamente copiados para o Amazon S3. O Amazon Redshift monitora continuamente a saúde do cluster e automaticamente replica os dados de unidades com falha e substitui os nós, conforme necessário. Consulte as Perguntas frequentes do Amazon Redshift²⁶ para obter mais informações.

Antipadrões

Como o Amazon Redshift é um sistema de gerenciamento de banco de dados relacional baseado em SQL (RDBMS), ele é compatível com outros aplicativos de RDBMS e ferramentas de inteligência empresarial. Embora o Amazon Redshift forneça a funcionalidade de um RDBMS típico, incluindo funções de processamento de transações on-line (OLTP), ele não foi projetado para essas

cargas de trabalho. Se você espera uma carga de trabalho de alta simultaneidade que geralmente envolve ler e gravar todas as colunas para um pequeno número de registros por vez, considere usar o Amazon RDS ou o Amazon DynamoDB.

Pesquisar

Os aplicativos que exigem uma funcionalidade de pesquisa sofisticada geralmente superam os recursos dos bancos de dados relacionais ou NoSQL. Um serviço de pesquisa pode ser usado para indexar e pesquisar o formato de texto livre e estruturado, e pode suportar funcionalidades que não estejam disponíveis em outros bancos de dados, como classificação de resultados personalizáveis, lapidação para filtragem, sinônimos, stemming, etc.

Na AWS, você tem a opção entre o Amazon CloudSearch e o Amazon Elasticsearch Service (Amazon ES). Por um lado, o Amazon CloudSearch é um serviço gerenciado que requer pouca configuração e será dimensionado automaticamente. Por outro lado, o Amazon ES oferece uma API de código aberto e proporciona mais controle sobre os detalhes da configuração. O Amazon ES também evoluiu para se tornar muito mais do que apenas uma solução de pesquisa. É frequentemente usado como um mecanismo de análise para casos de uso, como análise de log, monitoramento de aplicativo em tempo real e análise de fluxo de cliques.

Escalabilidade

O Amazon CloudSearch e o Amazon ES usam particionamento de dados e replicação para escalar horizontalmente. A diferença é que, com o Amazon CloudSearch, você não precisa se preocupar com o número de partições e réplicas necessárias porque o serviço lida com tudo isso automaticamente.

Alta disponibilidade

Ambos os serviços fornecem recursos que armazenam dados de forma redundante nas Zonas de disponibilidade. Para obter mais detalhes, consulte a documentação de cada serviço.

Removendo pontos únicos de falha

Os sistemas de produção normalmente vêm com objetivos definidos ou implícitos em termos de tempo de atividade. Um sistema é altamente disponível quando pode suportar a falha de um indivíduo ou vários componentes (por exemplo, discos rígidos, servidores, links de rede, etc.). Você pode pensar em maneiras para automatizar a recuperação e reduzir a interrupção em todas as camadas de sua arquitetura. Esta seção discute padrões de design de alta disponibilidade. Para obter mais detalhes sobre o assunto, consulte o whitepaper “Construindo aplicativos tolerantes a falhas” ²⁷.

Apresentando redundância

Pontos únicos de falha podem ser removidos pela introdução da redundância, que está tendo vários recursos para a mesma tarefa. A redundância pode ser implementada no modo de espera ou modo ativo.

Na *redundância de espera* quando um recurso falha, a funcionalidade é recuperada em um recurso secundário usando um processo chamado *failover*. O failover normalmente exigirá algum tempo antes de ser concluído e, durante esse período, o recurso permanecerá indisponível. O recurso secundário pode ser iniciado automaticamente somente quando necessário (para reduzir custos) ou pode já estar em execução ocioso (para acelerar o failover e minimizar a interrupção). A redundância em espera é geralmente usada para componentes stateful, como bancos de dados relacionais.

Na *redundância ativa*, as solicitações são distribuídas para vários recursos de computação redundantes e, quando uma delas falha, o restante pode simplesmente absorver uma parte maior da carga de trabalho. Em comparação com a redundância de espera, ela pode alcançar uma melhor utilização e afetar uma população menor quando houver uma falha.

Detectar falhas

Você deve procurar construir o máximo de automação possível tanto na detecção quanto na reação à falha. Você pode usar serviços como o ELB e o Amazon Route53 para configurar verificações de integridade e falhas de máscara, roteando o tráfego para endpoints íntegros. Além disso, o Auto Scaling pode ser configurado para substituir automaticamente os nós não íntegros. Você também pode substituir nós não íntegros usando o recurso de recuperação automática do Amazon EC2²⁸ ou serviços como o AWS OpsWorks e o AWS

Elastic Beanstalk. Não será possível prever todos os possíveis cenários de falha no primeiro dia. Certifique-se de coletar registros e métricas suficientes para entender o comportamento normal do sistema. Depois de entender isso, você poderá configurar alarmes para intervenção manual ou resposta automatizada.

Projetando boas verificações de integridade

A configuração das verificações de integridade certas para seu aplicativo determinará sua capacidade de responder correta e prontamente a vários cenários de falha. A especificação da verificação de integridade incorreta pode reduzir a disponibilidade do aplicativo.

Em um aplicativo típico de três camadas, você configura verificações de integridade no serviço Elastic Load Balancing. Projete suas verificações de integridade com o objetivo de avaliar com segurança a integridade dos nós de back-end. Uma verificação de integridade de TCP simples não detectaria o cenário em que a própria instância estivesse íntegra, mas o processo do servidor web tivesse falhado. Em vez disso, você deve avaliar se o servidor web pode retornar uma resposta HTTP 200 para alguma solicitação simples.

Nessa camada, pode não ser uma boa ideia configurar o que é chamado de verificação de integridade profunda, que é um teste que depende de outras camadas de seu aplicativo para obter êxito (isso pode resultar em falsos positivos). Por exemplo, se sua verificação de integridade também avaliar se a instância pode se conectar a um banco de dados de back-end, você corre o risco de marcar todos os seus servidores web como não íntegros, quando esse nó de banco de dados ficar indisponível por pouco tempo. Uma abordagem em camadas geralmente é a melhor. Uma verificação de integridade aprofundada pode ser adequada para implementar no nível Amazon Route53. Ao executar uma verificação mais holística que determina se esse ambiente é realmente capaz de fornecer a funcionalidade necessária, você pode configurar o Amazon Route53 para realizar failover em uma versão estática do seu site, até que seu banco de dados esteja ativo e em execução novamente.

Armazenamento físico de dados durável

Seu aplicativo e seus usuários criarão e manterão uma variedade de dados. É crucial que sua arquitetura proteja a disponibilidade e a integridade dos dados. A replicação de dados é a técnica que introduz cópias redundantes de dados. Ela pode ajudar a dimensionar horizontalmente a capacidade de leitura, mas também aumenta a durabilidade e a disponibilidade dos dados. A replicação pode ocorrer de alguns modos diferentes.

A *replicação síncrona* reconhece uma transação somente depois de ter sido armazenada de forma durável no local principal e em suas réplicas. É ideal para proteger a integridade de dados do evento de uma falha do nó primário. A replicação síncrona também pode dimensionar a capacidade de leitura para consultas que exigem dados mais atualizados (consistência forte). A desvantagem da replicação síncrona é que o nó primário é acoplado às réplicas. Uma transação não pode ser confirmada antes que todas as réplicas realizem a gravação. Isso pode comprometer o desempenho e a disponibilidade (especialmente em topologias executadas em conexões de rede não confiáveis ou de alta latência). Pelo mesmo motivo, não é recomendado manter muitas réplicas síncronas.

Durabilidade: Nenhum substituto para backups

Independentemente da durabilidade da sua solução, isso não substitui os backups. A replicação síncrona armazenará, de forma redundante, todas as atualizações em seus dados, mesmo aquelas resultantes de erros de software ou erros humanos. No entanto, especialmente para objetos armazenados no Amazon S3, você pode usar o versionamento²⁹ para preservar, recuperar e restaurar qualquer uma das suas versões. Com o versionamento, você pode se recuperar de ações não intencionais do usuário e de falhas de aplicativos.

A *replicação assíncrona* separa o nó primário de suas réplicas às custas da introdução do atraso de replicação. Isso significa que as alterações realizadas no nó primário não são refletidas imediatamente em suas réplicas. As réplicas assíncronas são usadas para dimensionar horizontalmente a capacidade de leitura do sistema de consultas que possam tolerar esse atraso de replicação. Ela também pode ser usada para aumentar a durabilidade dos dados quando alguma perda de transações recentes puder ser tolerada durante um failover. Por exemplo, você pode manter uma réplica assíncrona de um banco de dados em uma região AWS separada como uma solução de recuperação de desastres.

A *replicação baseada em quorum* combina replicação síncrona e assíncrona para superar os desafios dos sistemas de bancos de dados distribuídos em larga escala. A replicação de vários nós pode ser gerenciada definindo um número mínimo de nós que devem participar de uma operação de gravação bem-sucedida. Uma discussão detalhada de armazenamentos de dados distribuídos está além do escopo deste documento. Você pode consultar o whitepaper da Amazon Dynamo³⁰ para saber mais sobre um conjunto básico de princípios que podem resultar em um sistema de banco de dados ultrassensível e altamente confiável.

Durabilidade dos dados na prática

É importante entender onde cada tecnologia que você está usando se encaixa nesses modelos de armazenamento de dados. Seu comportamento durante vários cenários de failover ou de backup/restauração deve estar alinhado ao seu objetivo de ponto de recuperação (RPO) e ao seu objetivo de tempo de recuperação (RTO). Você precisa verificar quantos dados esperaria perder e com que rapidez conseguiria retomar as operações. Por exemplo, o mecanismo Redis para o Amazon ElastiCache oferece suporte à replicação com failover automático, mas a replicação do mecanismo Redis é assíncrona. Durante um failover, é altamente provável que algumas transações recentes sejam perdidas. No entanto, o Amazon RDS com seu recurso Multi-AZ, foi projetado para fornecer replicação síncrona para manter os dados no nó em espera atualizados com o principal.

Resiliência automatizada de vários centros de dados

Aplicativos essenciais aos negócios também precisarão de proteção contra cenários de interrupção que afetem muito mais do que apenas um único disco, servidor ou rack. Em uma infraestrutura tradicional, você normalmente teria um plano de recuperação de desastres para permitir um failover em um segundo datacenter distante, caso houvesse uma grande interrupção na central principal. Devido à longa distância entre os dois datacenters, a latência torna impraticável a manutenção de cópias síncronas dos dados entre datacenters. Como resultado, um failover certamente levará à perda de dados ou a um processo de recuperação de dados muito caro. Torna-se um procedimento arriscado e nem sempre suficientemente testado. No entanto, este é um modelo que oferece excelente proteção contra uma probabilidade baixa, mas um enorme risco de impacto, por exemplo, uma catástrofe natural que destrói toda a sua infraestrutura por um longo tempo. Você pode consultar o whitepaper de recuperação de desastres da AWS para obter mais orientações sobre como implementar essa abordagem na AWS³¹.

Uma interrupção mais curta em um datacenter é um cenário mais provável. Para interrupções curtas, como a duração da falha não é prevista como longa, a escolha de realizar um failover é difícil e geralmente será evitada. Na AWS, é possível adotar uma proteção mais simples e eficiente contra esse tipo de falha. Cada região da AWS contém vários locais distintos chamados Zonas de disponibilidade. Cada Zona de disponibilidade foi projetada para ser isolada de falhas em outras Zonas de disponibilidade. Uma Zona de disponibilidade é um datacenter e, em alguns casos, uma Zona de disponibilidade consiste em vários

datacenters. Zonas de disponibilidade dentro de uma região fornecem conectividade de rede de baixa latência e de baixo custo para outras zonas na mesma região. Isso permite que você replique seus dados pelos datacenters de maneira síncrona, para que o failover possa ser automatizado e transparente para seus usuários.

Também é possível implementar a redundância ativa para que você não pague por recursos ociosos. Por exemplo, uma frota de servidores de aplicativos pode ser distribuída entre várias Zonas de disponibilidade e ser associada ao Elastic Load Balancing (ELB). Quando as instâncias do Amazon EC2 de uma determinada Zona de disponibilidade falharem nas verificações de integridade, o ELB interromperá o envio de tráfego para esses nós. Quando combinado com o Auto Scaling, o número de nós íntegros pode ser automaticamente rebalanceado para as outras Zonas de disponibilidade sem intervenção manual.

Na verdade, muitos dos serviços de nível superior na AWS são inerentemente projetados de acordo com o princípio Multi-AZ. Por exemplo, o Amazon RDS oferece alta disponibilidade e suporte de failover automático para instâncias de banco de dados usando implantações Multi-AZ, enquanto que com o Amazon S3 e o Amazon DynamoDB, seus dados são armazenados de forma redundante em várias instalações.

Isolamento de falhas e escalabilidade horizontal tradicional

Embora o padrão de redundância ativa seja ótimo para equilibrar o tráfego e lidar com instâncias ou interrupções da Zona de disponibilidade, não é suficiente se houver algo prejudicial sobre as solicitações em si. Por exemplo, pode haver cenários em que cada instância é afetada. Se uma solicitação específica acionar um bug que faz com que o sistema faça failover, o chamador poderá desencadear uma falha em cascata tentando repetidamente a mesma solicitação em todas as instâncias.

Fragmentação embaralhada

Uma melhoria de isolamento de falhas que pode ser feita na escala horizontal tradicional é chamada *fragmentação*. Semelhante à técnica tradicionalmente usada com sistemas de armazenamento de dados, em vez de espalhar o tráfego de todos os clientes em todos os nós, é possível agrupar as instâncias em fragmentos. Por exemplo, se tiver oito instâncias para seu serviço, você poderá criar quatro fragmentos de duas instâncias cada (duas instâncias para alguma

redundância em cada fragmento) e distribuir cada cliente para um fragmento específico. Dessa forma, é possível reduzir o impacto nos clientes em proporção direta ao número de fragmentos que você possui. No entanto, ainda haverá clientes afetados, portanto, a chave é tornar o cliente tolerante a falhas. Se o cliente puder experimentar todos os endpoints em um conjunto de recursos fragmentados, até que um seja bem-sucedido, você terá uma melhoria significativa. Essa técnica é chamada *fragmentação embaralhada*, e é descrito com mais detalhes na publicação do blog relevante³².

Otimizar o custo

Apenas movendo as arquiteturas existentes para a nuvem, as organizações podem reduzir as despesas de capital e gerar economias como resultado das economias de escala da AWS. Ao iterar e fazer uso de mais recursos da AWS, há mais oportunidades de criar arquiteturas de nuvem otimizadas para custos. Esta seção discute os princípios fundamentais de otimização de custos com a computação em nuvem da AWS.

Dimensionamento correto

A AWS oferece uma ampla variedade de tipos de recursos e configurações para atender a uma infinidade de casos de uso. Por exemplo, serviços como o Amazon EC2, o Amazon RDS, o Amazon Redshift e o Amazon Elasticsearch Service (Amazon ES) oferecem muitas opções de tipos de instâncias. Em alguns casos, você deve selecionar o tipo mais barato que atenda aos requisitos de sua carga de trabalho. Em outros casos, usar menos instâncias de um tipo de instância maior pode resultar em menor custo total ou melhor desempenho. Você deve avaliar e selecionar o tipo de instância correto, dependendo de como sua carga de trabalho utiliza CPU, RAM, rede, tamanho de armazenamento e E/S.

Da mesma forma, você pode reduzir os custos selecionando a solução de armazenamento certa para suas necessidades. Por exemplo, o Amazon S3 oferece uma variedade de classes de armazenamento, incluindo acesso padrão, redundância reduzida e acesso não frequente padrão. Outros serviços, como o Amazon EC2, o Amazon RDS e o Amazon ES, suportam diferentes tipos de volume do Amazon Elastic Block Store (Amazon EBS) (magnético, SSD de uso geral, SSD de IOPS provisionado) que você deve avaliar.

Monitoramento contínuo e marcação

A otimização de custos é um processo iterativo. Seu aplicativo e seu uso evoluirão com o tempo. Além disso, a AWS repete com frequência e regularmente libera novas opções.

A AWS fornece ferramentas³³ para ajudá-lo a identificar as oportunidades de economia de custos e manter seus recursos de tamanho adequado. Para tornar os resultados dessas ferramentas fáceis de interpretar, você deve definir e implementar uma política de marcação para seus recursos da AWS. Você pode tornar a marcação parte de seu processo de criação e automatizá-la com ferramentas de gerenciamento da AWS, como o AWS Elastic Beanstalk e o AWS OpsWorks. Você também pode usar as regras gerenciadas fornecidas pelo AWS Config para avaliar se as tags específicas são aplicadas aos seus recursos ou não.

Elasticidade

Outra maneira de economizar dinheiro com a AWS é aproveitando a elasticidade da plataforma. Planeje implementar o Auto Scaling para o máximo possível de cargas de trabalho do Amazon EC2, para que você dimensione horizontalmente quando necessário, e diminua a escala e reduza automaticamente seus gastos quando não precisar mais dessa capacidade. Além disso, você pode automatizar o desligamento de cargas de trabalho de não produção quando não estiver em uso³⁴. Por fim, considere quais cargas de trabalho de computação você poderia implementar no AWS Lambda para nunca pagar por recursos ociosos ou redundantes.

Sempre que possível, substitua as cargas de trabalho do Amazon EC2 por serviços gerenciados da AWS que não exijam a tomada de decisões de capacidade (por exemplo, ELB, Amazon CloudFront, Amazon SQS, Amazon Kinesis Firehose, AWS Lambda, Amazon SES, Amazon CloudSearch) para modificar facilmente a capacidade como e quando necessário (por exemplo, Amazon DynamoDB, Amazon RDS e Amazon Elasticsearch Service).

Aproveite a variedade de opções de compra

O preço da instância sob demanda do Amazon EC2 oferece flexibilidade máxima sem confirmação de longo prazo. Há mais duas maneiras de pagar por instâncias do Amazon EC2 que podem ajudar a reduzir gastos: Instâncias reservadas e instâncias spot.

Capacidade reservada

As instâncias reservadas do Amazon EC2 permitem reservar a capacidade de computação do Amazon EC2 em troca de uma taxa horária com desconto significativo em comparação com o preço de instâncias sob demanda. Isso é ideal para aplicativos com requisitos mínimos de capacidade previsíveis. É possível aproveitar as ferramentas como os relatórios de uso do AWS Trusted Advisor ou do Amazon EC2 para identificar os recursos de computação que você usa na maior parte do tempo, e que deve considerar como reserva. Dependendo das suas compras por instância reservada, os descontos serão refletidos na fatura mensal. Observe que não há diferença técnica entre uma instância sob demanda do EC2 e uma instância reservada. A diferença está na maneira como você paga pelas instâncias que reserva.

Opções de capacidade reservada também existem para outros serviços (por exemplo, Amazon Redshift, Amazon RDS, Amazon DynamoDB e Amazon CloudFront).

Dica: *Você não deve se comprometer com as compras de instância reservada antes de fazer uma análise comparativa suficiente em seu aplicativo na produção. Depois de adquirir a capacidade reservada, é possível usar os relatórios de utilização de instância reservada para garantir que você ainda aproveite ao máximo sua capacidade reservada.*

Instâncias spot

Para cargas de trabalho menos constantes, você pode considerar o uso de instâncias spot. As instâncias spot do Amazon EC2 permitem oferecer uma capacidade de computação sobressalente do Amazon EC2. Como as instâncias spot geralmente estão disponíveis com um desconto em comparação com os preços sob demanda, você pode reduzir significativamente o custo de execução de seus aplicativos.

As instâncias spot são ideais para cargas de trabalho que têm hora de início e de fim flexíveis. Sua instância spot é iniciada quando seu lance excede o preço atual de mercado spot, e continuará sendo executado até você optar por encerrá-lo ou até que o preço de mercado spot exceda seu lance. Se o preço de mercado spot aumentar acima do seu preço de oferta, sua instância será encerrada automaticamente e você não será cobrado pela hora parcial em que sua instância foi executada.

Como resultado, as instâncias spot são ótimas para cargas de trabalho que tenham tolerância à interrupção. No entanto, você também pode usar instâncias spot quando precisar de disponibilidade mais previsível:

Estratégia de licitação: É cobrado o preço de mercado spot (não o seu preço de oferta) enquanto a instância spot for executada. Sua estratégia de lances poderia ter um lance muito mais alto do que isso, com a expectativa de que, mesmo que o preço de mercado ocasionalmente aumente, você ainda estará economizando um grande custo a longo prazo.

Misture com sob demanda: Considere a possibilidade de combinar instâncias reservadas, sob demanda e pontuais para combinar uma capacidade mínima previsível com o acesso “oportunista” a recursos de computação adicionais, dependendo do preço de mercado spot. Esta é uma ótima maneira de melhorar o rendimento ou o desempenho do aplicativo.

Blocos spot para cargas de trabalho de duração definida: Você também pode definir lances para instâncias spot de duração fixa. Estes têm diferentes preços por hora, mas permitem especificar um requisito de duração. Se seu lance for aceito, sua instância continuará a ser exibida até você optar por encerrá-la, ou até que a duração especificada seja encerrada; sua instância não será encerrada devido a alterações no preço spot (mas, é claro, você ainda deve projetar para tolerância a falhas porque uma instância spot ainda pode falhar como qualquer outra instância do EC2).

Melhores práticas de preços spot

Instâncias spot permitem que você lance em vários tipos de instância simultaneamente. Como os preços flutuam de forma independente para cada tipo de instância em uma Zona de disponibilidade, frequentemente, é possível obter mais capacidade de computação pelo mesmo preço se o aplicativo for projetado para ser flexível sobre os tipos de instância. Teste seu aplicativo em diferentes tipos de instâncias, quando possível. Faça lances para todos os tipos de instância que atendam aos seus requisitos para reduzir ainda mais os custos.

Armazenamento em cache

O armazenamento em cache é uma técnica que armazena dados calculados anteriormente para uso futuro. Essa técnica é usada para melhorar o desempenho do aplicativo e aumentar a eficiência de custo de uma implementação. Pode ser aplicado em várias camadas de uma arquitetura de TI.

Cache de dados de aplicativos

Os aplicativos podem ser projetados para armazenar e recuperar informações de caches de memória rápidos e gerenciados. As informações armazenadas em cache podem incluir os resultados de consultas ao banco de dados com utilização intensiva de E/S ou o resultado do processamento intensivo de computação. Quando o conjunto de resultados não é encontrado no cache, o aplicativo pode calculá-lo ou recuperá-lo de um banco de dados e armazená-lo em cache para solicitações subsequentes. No entanto, quando um conjunto de resultados é encontrado no armazenamento em cache, o aplicativo pode usá-lo diretamente, o que melhora a latência para os usuários finais e reduz a carga nos sistemas de back-end. Seu aplicativo pode controlar por quanto tempo cada item em cache permanecerá válido. Em alguns casos, até mesmo alguns segundos de armazenamento em cache para objetos muito populares pode resultar em uma redução drástica na carga do banco de dados.

O Amazon ElastiCache é um serviço web que facilita implantar, operar e escalar um cache na memória na nuvem. Ele suporta dois mecanismos de armazenamento em cache de memória de código aberto: Memcached e Redis. Para obter mais detalhes sobre como selecionar o mecanismo certo para sua carga de trabalho, bem como uma descrição dos padrões de design comuns do ElastiCache, consulte o whitepaper sobre “Desempenho em escala com o Amazon ElastiCache”.³⁵

Cache de ponto

Cópias de conteúdo estático (por exemplo, imagens, arquivos css, streaming de vídeo pré-gravado) e conteúdo dinâmico (por exemplo, resposta em html, vídeo ao vivo) podem ser armazenadas em cache no Amazon CloudFront, que é uma rede de entrega de conteúdo (CDN) que consiste de vários pontos de presença ao redor do mundo. O cache de borda permite que o conteúdo seja atendido por uma infraestrutura mais próxima dos visualizadores, reduzindo a latência e oferecendo as taxas de transferência de dados altas e sustentadas necessárias para entregar grandes objetos populares aos usuários finais em grande escala.

As solicitações para o seu conteúdo são levadas de volta ao Amazon S3 ou aos seus servidores de origem. Se a origem estiver em execução na AWS, as solicitações serão transferidas por caminhos de rede otimizados para uma experiência mais confiável e consistente. O Amazon CloudFront pode ser usado para fornecer todo o seu site, incluindo conteúdo sem armazenamento de cache. O benefício nesse caso é que o Amazon CloudFront reutiliza as conexões existentes entre o ponto do Amazon CloudFront e o servidor de origem, reduzindo a latência de configuração de conexão para cada solicitação de origem. Outras otimizações de conexão também são aplicadas para evitar gargalos na Internet e utilizar totalmente a largura de banda disponível entre o ponto de presença e o visualizador. Isso significa que o Amazon CloudFront pode agilizar a entrega de seu conteúdo dinâmico e fornecer aos visualizadores uma experiência consistente e confiável, mas personalizada, ao navegar pelo seu aplicativo web. O Amazon CloudFront também aplica os mesmos benefícios de desempenho para fazer upload de solicitações como àquelas aplicadas às solicitações de download de conteúdo dinâmico.

Segurança

A maioria das ferramentas e técnicas de segurança com as quais você já pode estar familiarizado em uma infraestrutura de TI tradicional pode ser usada na nuvem. Ao mesmo tempo, a AWS permite melhorar sua segurança de várias maneiras. A AWS é uma plataforma que permite formalizar o design de controles de segurança na própria plataforma. Ela simplifica o uso do sistema para administradores e para aqueles que executam a TI, e facilita muito o seu ambiente de auditoria de maneira contínua. Esta seção fornece uma visão geral de alto nível das melhores práticas de segurança da AWS. Para uma visão detalhada de como você pode alcançar um alto nível de governança de segurança, consulte a seção “Segurança em escala: Whitepapers de “Governança na AWS”³⁶ e as “Melhores práticas de segurança da AWS”³⁷.

Utilize os recursos da AWS para defesa profunda

A AWS fornece diversos recursos que podem ajudar os arquitetos a criar defesa profunda. A partir do nível de rede, você pode criar uma topologia de VPC que isola partes da infraestrutura por meio do uso de sub-redes, grupos de segurança e controles de roteamento. Serviços como o AWS WAF, um firewall de aplicativos web, podem ajudar a proteger seus aplicativos web contra injeção de SQL e outras vulnerabilidades no código do seu aplicativo. Para controle de acesso, você

pode usar o IAM para definir um conjunto granular de políticas e atribuí-las a usuários, grupos e recursos da AWS. Por fim, a plataforma da AWS oferece uma ampla variedade de opções para proteção de dados, seja em trânsito ou em repouso com criptografia³⁸. Uma lista exaustiva de todos os recursos de segurança está além do escopo deste documento, mas você pode saber mais visitando a página de Segurança da AWS³⁹.

Transferência de responsabilidade de segurança para a AWS

A AWS opera sob um modelo de responsabilidade de segurança compartilhada, em que a AWS é responsável pela segurança da infraestrutura de nuvem subjacente, e você é responsável por proteger as cargas de trabalho que implanta na AWS. Dessa forma, você pode reduzir o escopo de sua responsabilidade e focar em suas principais competências por meio do uso de serviços gerenciados da AWS. Por exemplo, quando você usa serviços como o Amazon RDS, o Amazon ElastiCache, o Amazon CloudSearch etc., os patches de segurança se tornam responsabilidade da AWS. Isso não apenas reduz a sobrecarga operacional de sua equipe, mas também pode reduzir sua exposição a vulnerabilidades.

Reduza o acesso privilegiado

Quando você trata os servidores como recursos programáveis, também pode aproveitar isso para obter benefícios no espaço de segurança. Quando puder alterar seus servidores sempre que precisar, você poderá eliminar a necessidade de acesso do sistema operacional convidado a ambientes de produção. Se uma instância tiver um problema, você poderá rescindi-la ou substituí-la automática ou manualmente. No entanto, antes de substituir as instâncias, você deve coletar e armazenar centralmente os registros em suas instâncias que podem ajudar a recriar problemas em seu ambiente de desenvolvimento e implantá-los como correções por meio do processo de implantação contínua. Isso é particularmente importante em um ambiente de computação elástico em que os servidores são temporários. Você pode usar o Amazon CloudWatch logs para coletar essas informações. Onde você não tem acesso e precisa, pode implementar o acesso a tempo (just-in-time) usando uma ação de API para abrir a rede para gerenciamento somente quando necessário. Você pode integrar essas solicitações de acesso ao seu sistema de tíquetes, para que as solicitações de acesso sejam rastreadas e tratadas dinamicamente somente após a aprovação.

Outra fonte comum de risco de segurança é o uso de contas de serviço. Em um ambiente tradicional, as contas de serviço geralmente recebem credenciais de longo prazo armazenadas em um arquivo de configuração. Na AWS, você pode usar funções do IAM para conceder permissões a aplicativos em execução em instâncias do Amazon EC2 por meio do uso de credenciais de curto prazo. Essas credenciais são automaticamente distribuídas e rotacionadas. Para aplicativos móveis, o uso do Amazon Cognito permite que dispositivos clientes obtenham acesso controlado a recursos da AWS por meio de tokens temporários. Para usuários do Console de Gerenciamento da AWS, você pode fornecer acesso federado por meio de tokens temporários em vez de criar usuários do IAM em sua conta da AWS. Dessa forma, um funcionário que sair da sua organização e for removido do diretório de identidade da organização também perderá o acesso à sua conta da AWS.

Segurança como código

Estruturas de segurança tradicionais, regulamentos e políticas organizacionais definem os requisitos de segurança relacionados a itens como regras de firewall, controles de acesso à rede, sub-redes internas/externas e proteção do sistema operacional. Você pode implementá-las também em um ambiente da AWS, mas agora tem a oportunidade de capturá-las em um script que define um “ambiente dourado”. Isso significa que você pode criar um script do AWS CloudFormation que capture sua política de segurança e implante-a de maneira confiável. As melhores práticas de segurança agora podem ser reutilizadas em vários projetos e se tornar parte de pipeline de integração contínua. Você pode realizar testes de segurança como parte de seu ciclo de lançamento e descobrir automaticamente as lacunas de aplicativos e desvios da sua política de segurança.

Além disso, para maior controle e segurança, os modelos do AWS CloudFormation podem ser importados como “produtos” no AWS Service Catalog.⁴⁰ Isso permite o gerenciamento centralizado de recursos para suportar requisitos consistentes de governança, segurança e conformidade, ao mesmo tempo em que permite que os usuários implantem rapidamente apenas os serviços de TI aprovados de que precisam. Você aplica as permissões do IAM para controlar quem pode visualizar e modificar seus produtos e define restrições para restringir as maneiras pelas quais recursos específicos da AWS podem ser implantados para um produto.

Auditoria em tempo real

Testar e auditar seu ambiente é fundamental para se movimentar rapidamente e permanecer seguro. Abordagens tradicionais que envolvem verificações periódicas (e frequentemente manuais ou baseadas em amostras) não são suficientes, especialmente em ambientes ágeis onde a mudança é constante. Na AWS, é possível implementar monitoramento e automação contínuos de controles para minimizar a exposição a riscos de segurança. Serviços como o AWS Config, o Amazon Inspector e o AWS Trusted Advisor monitoram continuamente a conformidade ou as vulnerabilidades, fornecendo uma visão clara de quais recursos de TI estão em conformidade e quais não estão. Com as regras do AWS Config, você também saberá se algum componente ficou fora de conformidade mesmo por um breve período de tempo, tornando as auditorias do ponto no tempo e do período no tempo muito eficazes. Você pode implementar registros extensivos para seus aplicativos (usando Amazon CloudWatch logs) e para as chamadas reais da API da AWS, ativando o AWS CloudTrail.⁴¹ O AWS CloudTrail é um serviço web que registra chamadas de API para serviços suportados da AWS em sua conta da AWS e entrega um arquivo de registro ao seu bucket do Amazon S3. Os registros podem ser armazenados de forma imutável e processados automaticamente para notificar ou até mesmo tomar medidas em seu nome, protegendo sua organização da não conformidade. Você pode usar o AWS Lambda, o Amazon EMR, o Amazon Elasticsearch Service ou ferramentas de terceiros do AWS Marketplace para verificar registros para detectar itens como permissões não utilizadas, uso excessivo de contas privilegiadas, uso de chaves, logins anômalos, violações de diretivas e abuso do sistema.

Conclusão

Este whitepaper fornece orientação para projetar arquiteturas que aproveitem ao máximo a plataforma da AWS cobrindo princípios importantes e padrões de design: de como selecionar o banco de dados certo para seu aplicativo, para arquitetar aplicativos que possam escalar horizontalmente e com alta disponibilidade. Como cada caso de uso é único, você terá que avaliar como eles podem ser aplicados à sua implementação. O tópico das arquiteturas de computação em nuvem é amplo e está continuamente evoluindo. A partir de agora, você pode ficar atualizado com a riqueza de material disponível no site da AWS e as ofertas de treinamento e certificação da AWS.

Contribuidores

O seguinte indivíduo contribuiu para este documento:

- Andreas Chatzakis, gerente, Soluções de arquitetura da AWS

Outras leituras

Para mais exemplos de arquitetura, você pode consultar o Centro de Arquitetura da AWS⁴². Para aplicativos já em execução na AWS, recomendamos que você também leia o whitepaper “AWS Well Architected Framework”⁴³ que complementa este documento fornecendo um modelo de avaliação estruturado. Por fim, para validar sua prontidão operacional, você também pode consultar a lista de verificação abrangente da AWS⁴⁴.

Observações

- ¹ Sobre a AWS: <https://aws.amazon.com/about-aws/>
- ² A infraestrutura global da AWS: <https://aws.amazon.com/about-aws/global-infrastructure/>
- ³ Por exemplo, há o manipulador de sessão do PHP Amazon DynamoDB (<http://docs.aws.amazon.com/aws-sdk-php/v3/guide/service/dynamodb-session-handler.html>) e o manipulador de sessão Tomcat do Amazon DynamoDB <http://docs.aws.amazon.com/AWSSdkDocsJava/latest/DeveloperGuide/java-dg-tomcat-session-manager.html>
- ⁴ Sticky session de ELB <http://docs.aws.amazon.com/ElasticLoadBalancing/latest/DeveloperGuide/elb-sticky-sessions.html>
- ⁵ Whitepaper sobre “Opções de análise de big data na AWS” https://do.awsstatic.com/whitepapers/Big_Data_Analytics_Options_on_AWS.pdf
- ⁶ Bootstrapping com scripts de dados do usuário e cloud-init: <http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-metadata.html>
- ⁷ Eventos do ciclo de vida do AWS Opsworks <http://docs.aws.amazon.com/opsworks/latest/userguide/workingcookbook-events.html>
- ⁸ Recursos personalizados do CloudFormation suportados pelo AWS Lambda: <http://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-custom-resources-lambda.html>
- ⁹ Amazon Machine Images <http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html>
- ¹⁰ AMIs para os tempos de operação do AWS Elastic Beanstalk: <http://docs.aws.amazon.com/elasticbeanstalk/latest/dg/concepts.platforms.html>
- ¹¹ Customização do AWS Elastic Beanstalk com arquivos de configuração: <http://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ebextensions.html>
- ¹² AWS Elastic Beanstalk: <https://aws.amazon.com/elasticbeanstalk/>

- ¹³ Recuperação automática do Amazon EC2:
<http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-recover.html>
- ¹⁴ Auto Scaling: <https://aws.amazon.com/autoscaling/>
- ¹⁵ Alarmes do Amazon CloudWatch:
<http://docs.aws.amazon.com/AmazonCloudWatch/latest/DeveloperGuide/AlarmThatSendsEmail.html>
- ¹⁶ Eventos do Amazon CloudWatch:
<http://docs.aws.amazon.com/AmazonCloudWatch/latest/DeveloperGuide/WhatIsCloudWatchEvents.html>
- ¹⁷ Ciclo de vida do AWS OpsWorks
<http://docs.aws.amazon.com/opsworks/latest/userguide/workingcookbook-events.html>
- ¹⁸ Eventos programados do AWS Lambda:
<http://docs.aws.amazon.com/lambda/latest/dg/with-scheduled-events.html>
- ¹⁹ Recuo exponencial e tremulação
<http://www.awsarchitectureblog.com/2015/03/backoff.html>
- ²⁰ Você pode ver a lista completa de produtos da AWS aqui:
<http://aws.amazon.com/products/>
- ²¹ Whitepaper sobre “Arquiteturas multicamadas sem servidor da AWS”
https://do.awsstatic.com/whitepapers/AWS_Serverless_Multi-Tier_Architectures.pdf
- ²² Melhores práticas para o Amazon RDS:
http://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_BestPractices.html
- ²³ Bancos de dados NoSQL na AWS <https://aws.amazon.com/nosql/>
- ²⁴ Melhores práticas para o Amazon DynamoDB:
<http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/BestPractices.html>
- ²⁵ Melhores práticas para migrar do RDBMS para o Amazon DynamoDB:
<https://do.awsstatic.com/whitepapers/migration-best-practices-rdbms-to-dynamodb.pdf>
- ²⁶ Perguntas frequentes sobre o Amazon Redshift:
<https://aws.amazon.com/redshift/faqs/>

- ²⁷ Whitepaper sobre “Construindo aplicativos tolerantes a falhas”:
<https://do.awsstatic.com/whitepapers/aws-building-fault-tolerant-applications.pdf>
- ²⁸ Recupere sua instância:
<http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-recover.html>
- ²⁹ Versionamento do Amazon S3:
<http://docs.aws.amazon.com/AmazonS3/latest/dev/Versioning.html>
- ³⁰ “Dynamo: Loja de valor-chave altamente disponível da Amazon”
http://www.allthingsdistributed.com/2007/10/amazons_dynamo.html
- ³¹ “Usando a Amazon Web Services para recuperação de desastres”
https://media.amazonwebservices.com/AWS_Disaster_Recovery.pdf
- ³² Fragmentação embaralhada
<http://www.awsarchitectureblog.com/2014/04/shuffle-sharding.html>
- ³³ Monitorando uso e custos
<http://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/monitoring-costs.html>
- ³⁴ Crie alarmes que interrompam ou terminem uma instância
<http://docs.aws.amazon.com/AmazonCloudWatch/latest/DeveloperGuide/UsingAlarmActions.html>
- ³⁵ “Desempenho em escala com o Amazon ElastiCache”:
<https://do.awsstatic.com/whitepapers/performance-at-scale-with-amazon-elasticache.pdf>
- ³⁶ “Segurança em escala: Governança na AWS”
https://do.awsstatic.com/whitepapers/compliance/AWS_Security_at_Scale_Governance_in_AWS_Whitepaper.pdf
- ³⁷ “Melhores práticas de segurança da AWS”:
<https://do.awsstatic.com/whitepapers/aws-security-best-practices.pdf>
- ³⁸ Protegendo dados em repouso com criptografia:
<https://do.awsstatic.com/whitepapers/aws-securing-data-at-rest-with-encryption.pdf>
- ³⁹ Segurança da AWS: <http://aws.amazon.com/security>
- ⁴⁰ AWS Service Catalog: <https://aws.amazon.com/servicecatalog/>

- ⁴¹ “Segurança em escala: Fazendo login na AWS”
https://do.awsstatic.com/whitepapers/compliance/AWS_Security_at_Scale_Locking_in_AWS_Whitepaper.pdf
- ⁴² Centro de arquitetura da AWS <https://aws.amazon.com/architecture>
- ⁴³ “AWS Well Architected Framework”:
http://do.awsstatic.com/whitepapers/architecture/AWS_Well-Architected_Framework.pdf
- ⁴⁴ Lista de verificação operacional da AWS
http://media.amazonwebservices.com/AWS_Operational_Checklists.pdf